

Минобрнауки России
Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет
«Московский институт электронной техники»

Расчетно-Графическая работа по дисциплине “Численные Методы”

по теме
“Решение систем линейных алгебраических уравнений методом Якоби”

Выполнил: Максимов Глеб
Группа: РТ-22

Содержание

Основные понятия, постановка задачи	3
Системы Алгебраических Линейных Уравнений (СЛАУ)	3
Методы решения СЛАУ	3
Итерационные методы решения	4
Изложение алгоритма	5
Метод Якоби	5
Реализация алгоритма	6
Демонстрация алгоритма	9

• Системы Алгебраических Линейных Уравнений (СЛАУ)

[illegible]

где $\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{pmatrix}$, $\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}$, $\mathbf{f} = \begin{pmatrix} f_1 \\ f_2 \\ \vdots \\ f_n \end{pmatrix}$

$$x_i = \frac{\Delta_i}{\Delta},$$

$$\left(\begin{array}{cccc|c} a_{11} & a_{12} & \cdots & a_{1n} & f_1 \\ a_{21} & a_{12} & \cdots & a_{1n} & f_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} & f_n \end{array} \right)$$

$$\left(\begin{array}{cccc|c} a_{11} & a_{12} & \dots & a_{1n} & f_1 \\ 0 & a_{22}^{(1)} & \dots & a_{2n}^{(1)} & f_2^{(1)} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & a_{nn}^{(n-1)} & f_n^{(n-1)} \end{array} \right)$$

В обратном ходе переменные вычисляются последовательно начиная с последнего уравнения

$$x_n = \frac{f_n}{a_{nn}}, x_{n-1} = \frac{f_{n-1} - a_{n-1,n} \cdot x_n}{a_{n-1,n-1}}, \dots$$

- **Итерационные методы решения**

Рассматриваем системы вида $\mathbf{Ax} = \mathbf{f}$. Различные варианты итерационных методов связаны с переходом к эквивалентной системе

$$\mathbf{Ax} = \mathbf{f} \Leftrightarrow \mathbf{x} = \mathbf{Px} + \mathbf{g}$$

Берём некоторое начальное приближение $\mathbf{x}^{(0)}$ и очевидным образом строим итерационный процесс

$$\mathbf{x}^{(k)} = \mathbf{Px}^{(k-1)} + \mathbf{g}$$

Здесь $\mathbf{x}^{(k)}$ обозначает k -е по счету приближение к искомому вектору $\mathbf{x}$. Далее рассмотрим метод Якоби.

Изложение алгоритма

Метод Якоби

Запишем каждое уравнение системы $\mathbf{Ax} = \mathbf{f}$ в виде, разрешённом относительно неизвестного с коэффициентом на главной диагонали матрицы $\mathbf{A}$:

$$x_m = \frac{1}{a_{mm}}(f_m - a_{m1}x_1 - \dots - a_{m,m-1}x_{m-1} - a_{m,m+1}x_{m+1} - \dots - a_{mn}x_n),$$

$$m = 1, 2, \dots, n$$

То есть мы переписали $\mathbf{Ax} = \mathbf{f}$ в виде $\mathbf{x} = \mathbf{Px} + \mathbf{g}$ с матрицей

$$P = - \begin{pmatrix} 0 & \frac{a_{12}}{a_{11}} & \frac{a_{13}}{a_{11}} & \dots & \frac{a_{1n}}{a_{11}} \\ \frac{a_{21}}{a_{22}} & 0 & \frac{a_{23}}{a_{22}} & \dots & \frac{a_{2n}}{a_{22}} \\ \vdots & & & & \vdots \\ \frac{a_{n1}}{a_{nn}} & \frac{a_{n2}}{a_{nn}} & \dots & \frac{a_{nn}}{a_{nn}} & 0 \end{pmatrix}$$

Если ввести в рассмотрение диагональную матрицу

$$D = \begin{pmatrix} a_{11} & & & 0 \\ & a_{22} & & \\ & & \ddots & \\ 0 & & & a_{nn} \end{pmatrix},$$

то $P = -D^{-1}(A - D)$, $\mathbf{g} = D^{-1}\mathbf{f}$.

Итерационный процесс $\mathbf{x}^{(k)} = P\mathbf{x}^{(k-1)} + \mathbf{g}$ определённый с таким образом матрицей P называется *методом Якоби*. Фактически вычисления проводят по формулам

$$x_m^{(k)} = \frac{1}{a_{mm}}(f_m - a_{m1}x_1^{(k-1)} - \dots - a_{m,m-1}x_{m-1}^{(k-1)} - a_{m,m+1}x_{m+1}^{(k-1)} - \dots - a_{mn}x_n^{(k-1)}),$$

$$m = 1, 2, \dots, n$$

Для сходимости метода Якоби достаточно, чтобы для исходной матрицы $\mathbf{A}$ имело место диагональное преобладание, т.е. чтобы коэффициенты исходных уравнений удовлетворяли условиям

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}| \text{ для всех } i.$$

Реализация алгоритма

Рассмотрим реализацию алгоритма на языке Python с использованием библиотеки Numpy.

Зададим матрицу **A** из случайных чисел размером 5x5 удовлетворяющую условию сходимости метода и столбец свободных членов:

```
A = np.round((np.random.rand(5, 5) * 10) - 5 + 20* np.eye(5,5), 3)
f = np.round((np.random.rand(5) * 10 - 5), 3)
```

A

```
array([[19.17 ,  2.203, -4.999, -1.977, -3.532],
       [-4.077, 16.863, -1.544, -1.032,  0.388],
       [-0.808,  1.852, 17.045,  3.781, -4.726],
       [ 1.705, -0.827,  0.587, 16.404, -3.019],
       [ 3.007,  4.683, -1.866,  1.923, 23.764]])
```

f

```
array([ 3.946, -4.15 , -4.609, -3.302,  3.781])
```

Реализуем функцию проверки на метода сходимость:

```
def checkConvergence(A):
    n = A.shape[0]

    for i in range(n):
        sum = 0
        for a in np.abs(A[i, :]):
            sum += a

        sum -= np.abs(A[i, i])

        if A[i,i] <= sum:
            return False

    return True
```

Проверим систему на сходимость:

```
checkConvergence(A)
```

True

Реализуем функцию для решения системы методом Якоби:

```
def jacobiSolver(A, f, eps=1e-12, maxIters=1000):
    n = A.shape[0]

    x = np.zeros(n)

    for k in range(maxIters):
        x_new = np.zeros(n)

        for i in range(n):
            sum = 0

            for j in range(n):
                if j != i:
                    sum += A[i, j] * x[j]

            x_new[i] = (f[i] - sum) / A[i, i]

        if np.max(np.abs(x_new - x)) < eps:
            return x_new, k+1

    x = x_new

    return x, k+1
```

Найдём решение системы с помощью этой функции:

```
jacobiSolver(A, f)
```

```
(array([ 0.20791404, -0.22501991, -0.14227827, -0.19569615,  0.18180454]), 25)
```

Теперь зададим матрицу **A** которая не удовлетворяет условию сходимости метода:

```
A = np.round((np.random.rand(5, 5) * 10) - 5 + 0 * np.eye(5,5), 3)
f = np.round((np.random.rand(5) * 10 - 5), 3)
```

A

```
array([[ -4.017,  -0.789,   4.579,   0.332,   1.919],
       [ -1.845,   1.865,   3.346,  -4.817,   2.501],
       [  4.889,   2.482,  -2.196,   2.893,  -3.968],
       [ -0.521,   4.086,  -2.064,  -2.122,  -3.7   ],
       [ -4.806,   1.788,  -2.884,  -2.345,  -0.084]])
```

f

```
array([ -4.466,   0.741,  -3.533,   0.893,   1.998])
```

Проверим такую систему на сходимость:

```
checkConvergence(A)
```

False

Видим *False*, то есть метод не сойдется, проверяем:

```
jacobiSolver(A, f)
```

```
/tmp/ipykernel_319592/1548592516.py:14: RuntimeWarning: overflow encountered in
scalar multiply
    sum+=A[i, j]*x[j]
/tmp/ipykernel_319592/1548592516.py:14: RuntimeWarning: overflow encountered in
scalar add
    sum+=A[i, j]*x[j]
/tmp/ipykernel_319592/1548592516.py:16: RuntimeWarning: overflow encountered in
scalar divide
    x_new[i] = (f[i] - sum) / A[i, i]
/tmp/ipykernel_319592/1548592516.py:14: RuntimeWarning: invalid value encounter
ed in scalar add
    sum+=A[i, j]*x[j]
/tmp/ipykernel_319592/1548592516.py:18: RuntimeWarning: invalid value encounter
ed in subtract
    if np.max(np.abs(x_new - x)) < eps:
(array([nan, nan, nan, nan, nan]), 1000)
```

Как и ожидалось, за 1000 иттераций решение не было найдено.

Демонстрация алгоритма

